

NDNSF-DistributedRepo

A High-Availability NDN-Native Repository for NDNSF Applications

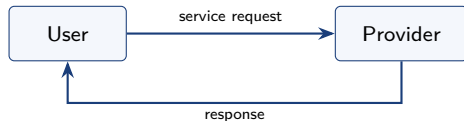
Tianxing Ma

Department of Computer Science
University of Memphis

July 2026

Why a Service-Aware Repository?

NDNSF invocation is intentionally short-lived



The request/ACK/selection/response path should not carry every large or durable object inline.

Applications need named data products that outlive one call

- ▶ Model and runtime artifacts
- ▶ UAV recordings, telemetry, and mission logs
- ▶ Intermediate data, configurations, and workflow records

Repo goal: keep these objects named, discoverable, replicated, and retrievable without teaching the framework their application meaning.

Evidence: [NDNSF-DistributedRepo/README.md](#), overview and Application Integration Status

Responsibility Boundary

Applications: meaning and workflow policy

DI model graphs, tensor dependencies, and scheduling; UAV video, MAVLink, mission, and operator UI



NDNSF-DistributedRepo: generic object and catalog policy

exact Data storage, manifests, placement, quorum writes, catalog, retention, tombstones, and repair



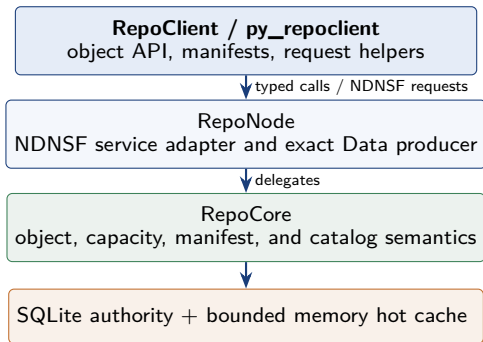
NDNSF Core: secure service invocation and cooperation

discovery, ACK/selection, permissions, NAC-ABE, one-time tokens, replay protection, large-data helpers

The repo is reusable because it understands **named objects and lifecycle metadata**, not AI models, videos, or missions.

Evidence: NDNSF-DistributedRepo/README.md, In-App Repo Plan and Application Integration Status

Layered Architecture



Evidence: RepoClient.hpp, RepoNode.hpp, RepoCore.hpp, RepoTypes.hpp

One durable state model, two exposure paths

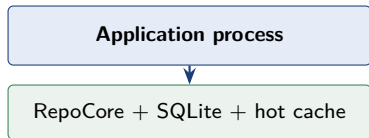
- ▶ Remote services use the normal NDNFSF network path.
- ▶ Trusted same-process callers use LocalServiceRegistry.
- ▶ Both paths invoke the same RepoCore semantics.
- ▶ Memory accelerates hot reads; SQLite remains authoritative.

Neither local invocation nor the hot cache creates a second object model or a memory-only durability mode.

Two Deployment Roles

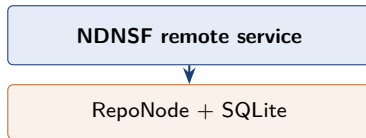
In-App Repo

- ▶ Embedded in a trusted application process
- ▶ SQLite-backed local objects and bounded hot cache
- ▶ Useful for artifacts, recordings, and intermediates
- ▶ Exposed through local services, remote services, or both



Persistent Repo

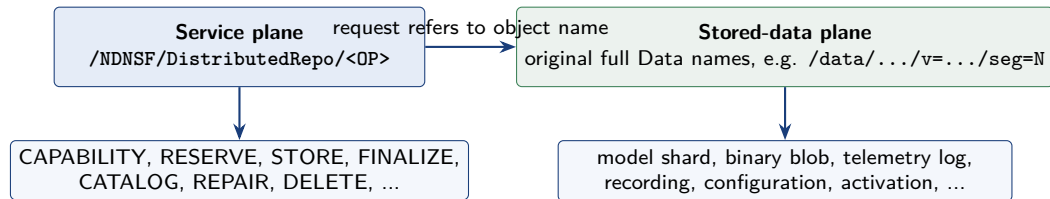
- ▶ Independent NDNFSF provider process
- ▶ Durable backup, shared catalog, and recovery role
- ▶ Normally accepts backup replicas
- ▶ SQLite backend remains fetchable after restart



Both roles use SQLite authority; deployment changes exposure and placement, not object semantics.

Evidence: README: In-App and Persistent Repo Modes; DistributedRepoNodeApp.cpp

Two Namespace Planes



- ▶ The shared service prefix tells NDNSF *what operation to invoke*.
- ▶ The original publisher-owned full name tells NDN *which exact Data packet is stored*; Repo does not rename or re-sign it.

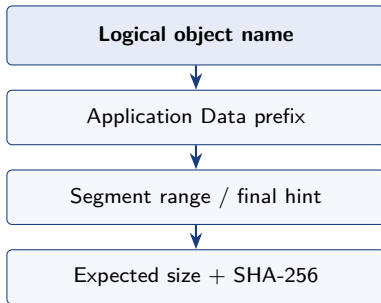
Evidence: README: Namespace and Trust Schema Design

Logical Object and Data Reference

RepoObjectManifest

Field	Purpose
objectName	Stable application object name
objectType	Generic application classification
sha256, size	End-to-end integrity metadata
segmentCount	Logical object layout
replicas	Desired factor and confirmed repo nodes
consistency	ONE, QUORUM, or ALL receipt threshold
lifecycle	STAGED, COMMITTED, or deleted
policyEpoch	Placement-policy version

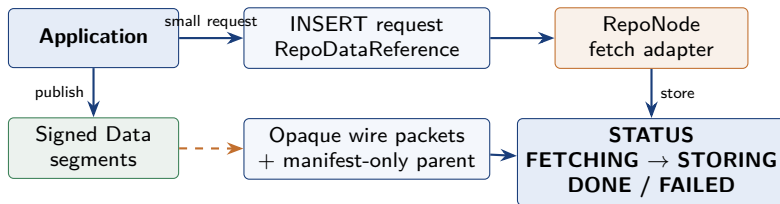
RepoDataReference



The reference lets a repo pull application-published signed Data without putting the large payload in the service request.

Evidence: RepoTypes.hpp: RepoObjectManifest and RepoDataReference

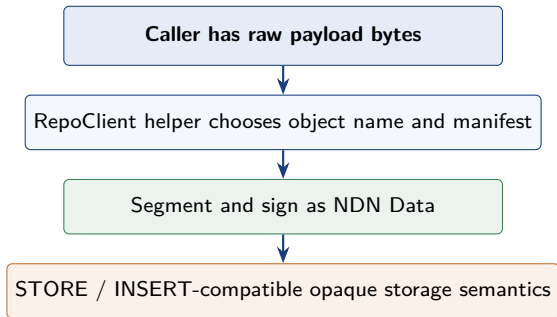
INSERT: Store App-Owned Segmented Data



The repo stores wire packets as opaque bytes. It does not decrypt, reinterpret, or re-authorize the application payload.

Evidence: `RepoNode::handleInsert`; README: App-Owned Segmented Data References

Payload APIs Are Adapters, Not Another Data Model



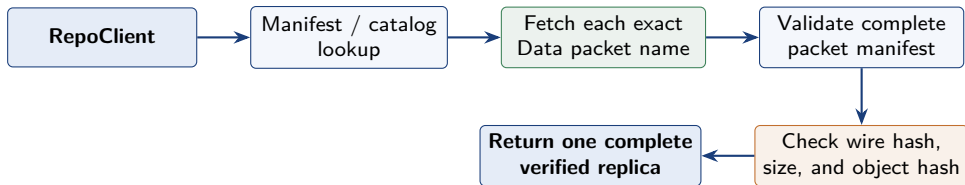
Why keep the adapters?

- ▶ Convenient for existing callers and tests
- ▶ Same logical object and manifest API
- ▶ Same opaque backend and catalog semantics

Large files use named segmented Data. Continuous video publication remains a stream problem, not a repo object problem.

Evidence: RepoClient::insertPayload, putSegmented, getObject; NDN SF large-data boundary

Fetch Exact Data, Fail Over by Replica

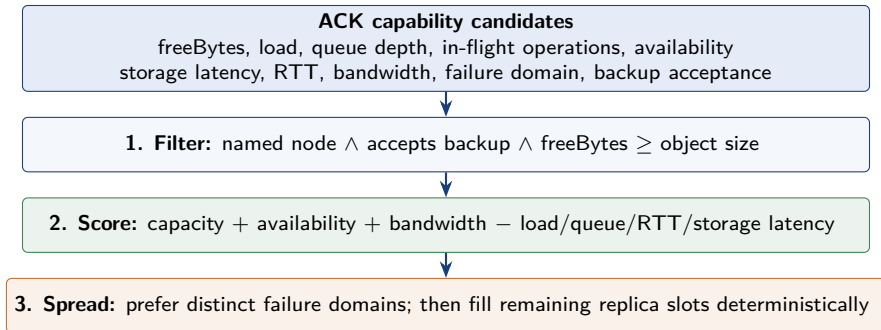


Repo responsibility: reject malformed/partial packet sets and retry the whole object at another replica.

Application responsibility: apply domain trust and decrypt or interpret the verified Data.

Evidence: `RepoClient::fetchSignedPackets`; exact-packet and replica-failover regressions

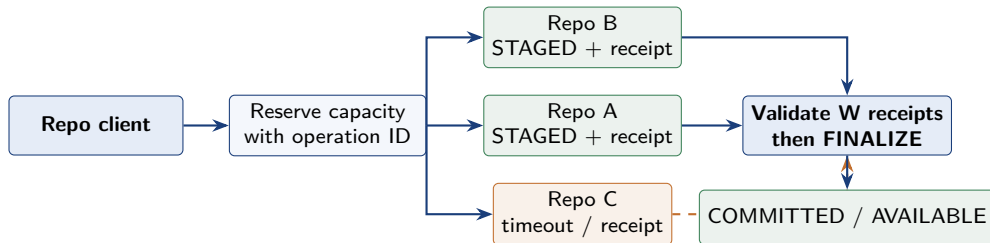
Capability-Driven Replica Placement



Placement uses live provider capability hints, then reserves capacity before sending replicated writes.

Evidence: RepoTypes.cpp: scoreCandidate and selectReplicas; py_repoclient capability helpers

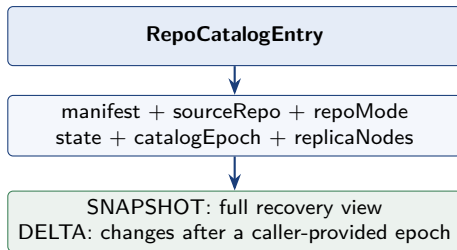
Targeted Parallel Write and Quorum Finalization



- ▶ RF is desired placement; W is required durable receipts.
- ▶ ONE, QUORUM, and ALL keep different failure/latency tradeoffs.
- ▶ Known replicas use secure Targeted calls in parallel.
- ▶ Below-quorum STAGED data is invisible to reads and repair.

Targeted reduces control rounds; authorization, one-time tokens, replay protection, and receipt validation remain active.

The Catalog Is Object-Level Metadata



Catalog operations

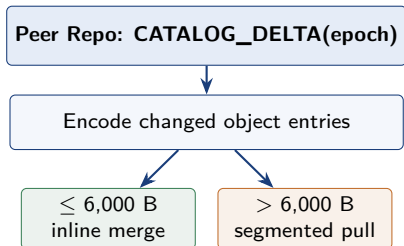
- ▶ **STATUS:** node mode, epoch, object count
- ▶ **LOOKUP:** one exact logical object
- ▶ **QUERY:** class, type, publisher, tags, state, metadata, time range

Catalogs exchange one record per logical object, not one record per Data segment.

Evidence: `RepoCatalogStatus/Entry/Delta`; README: Catalog and Replica Discovery

Decentralized Catalog Synchronization

Object-level anti-entropy



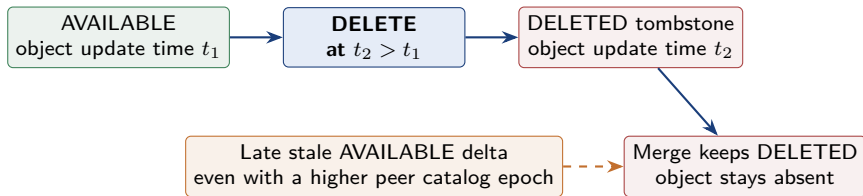
Measured recovered-sidecar path: six large pulls + two inline merges, 33 segments, zero fallback; merge time 5.200 s → 3.039 s.

Large-delta integrity path

1. Source publishes signed segmented Data under an exact versioned name.
2. One protected request binds name, schema, bytes, entries, and SHA-256.
3. Target fetches the complete object and validates it before merge.
4. Pull failure falls back to bounded inline batches.

Evidence: Spec 083; catalog_sync.py; CATALOG_MERGE_PULL MiniNDN treatment

Deletion Uses Tombstones, Not Silence



- ▶ Catalog sequence orders peer changes; object update time and delete semantics resolve object conflicts.
- ▶ Keeping the tombstone prevents an old replica advertisement from resurrecting removed data.

Evidence: generic object store tombstone gossip and epoch-conflict MiniNDN regressions

Retention and Durable Online Repair

Lifecycle policy

Condition	Catalog outcome
Expired	EXPIRED ; no repair
Live + under target	Create repair job
Otherwise	Healthy

Repair never uses STAGED, deleted, expired, conflicting, or hash-mismatched data as a source.

Sidecar runs repair outside handlers; durable leases and backlog survive restart.

Durable repair job

1. Scan creates idempotent jobs from catalog state.
2. Risk, priority, age, and replica deficit order claims.
3. Leased workers fetch exact Data; target verifies and stores.
4. Success completes; failure backs off.

Evidence: Specs 080–082; durable repair and phase telemetry

Measured Provider Failure and Recovery

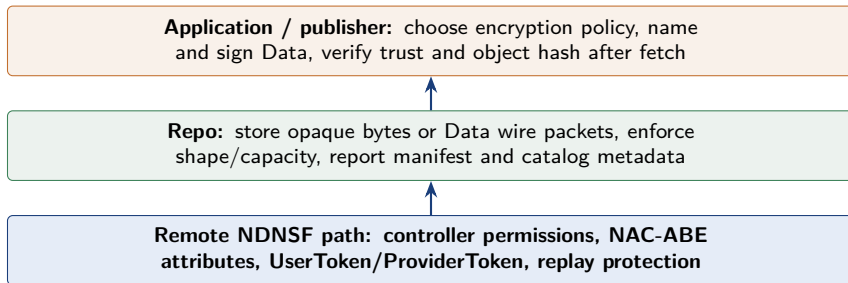


Matched 60 s campaign	Requests	Outage repair	Request p95
Quorum during failure	19/19 post-failure	later repairable	1,649.912 ms
Repair fast path	30/30	4/4	1,814.117 ms
Catalog segmented pull	30/30	4/4	1,779.222 ms

Latest fixed treatment: receipt floor $W=2$, zero invalid repair events, first repair 9.033 s after restart. This is diagnostic evidence, not a production SLO.

Evidence: Specs 079, 082, and 083 canonical MiniNDN summaries; seed 78004

Security and Trust Boundaries

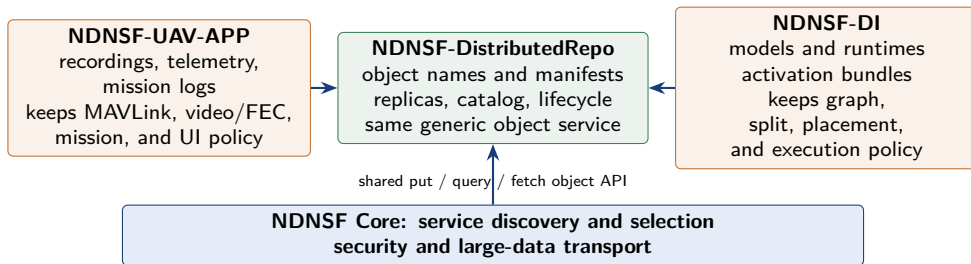


Trust schema: object Data names stay under the publisher identity; child certificates remain under parent namespaces.

Local path: trusted same-process composition only; it is never remotely selectable and never weakens remote authorization.

Evidence: README: Namespace and Trust Schema Design; RepoStoreBackend contract; NDNFS security invariants

One Generic Repo, Different Applications



The same object/catalog layer is useful across applications precisely because application semantics stay outside it.

Evidence: README: Application Integration Status; UAV and DI repo call sites

What Is Validated Today?

Implemented and exercised

- ▶ SQLite authority + hot cache across restart
- ▶ Exact signed Data + replica failover
- ▶ Parallel Targeted writes + quorum finalization
- ▶ Provider loss/restart + online repair
- ▶ Tombstones, conflicts, and segmented anti-entropy

```
sudo -E PYTHONPATH=... python3  
Experiments/NDNSF_DistributedRepo_  
Generic_Minindn.py
```

Remaining production work

- ▶ Larger sustained multi-node campaigns
- ▶ Crash-point/filesystem durability matrix
- ▶ WAN partition, churn, and rejoin
- ▶ Monitoring, quotas, and external receipts

Takeaway: exact Data stays on the data plane; authenticated control manages placement, lifecycle, and recovery.

Evidence: Specs 073–083; focused C++/Python/security regressions and canonical MiniNDN results